

Parallel and Distributed Processing

HPC Project Report — Laplace Solver

Student: Amna Nadeem

CMS: 404597

Course: Parallel and Distributed Processing

Instructor: Attique Dawood

Date: 13 May 2026

1. Introduction

The Laplace equation models steady-state heat diffusion on a 2D grid. Given a plate where the top edge is fixed at 100°C and all other edges at 0°C, the solver iteratively computes how heat spreads across the plate. Each interior cell is updated as the average of its four neighbours. This process is repeated for a fixed number of epochs until convergence.

Three implementations were developed: Serial, OpenMP, and MPI.

2. Part I — HPC Cluster Map

2.1 Cluster Overview

The NUST SINES HPC cluster was accessed via SSH at 10.19.10.150. Head node: master. Four compute nodes were discovered via `/etc/hosts`.

3. Part II — Laplace Solver Implementation

3.1 Serial

Standard double-nested loop over all interior cells each epoch. Used as the correctness baseline — all parallel versions must match center values.

3.2 OpenMP

The outer row loop is parallelised using:

```
#pragma omp parallel for schedule(static)
```

All threads share memory. No inter-thread communication is needed. Thread count is passed as a command-line argument.

3.3 MPI

The grid is split into horizontal strips using domain decomposition. Each process owns N/size rows and computes only its local portion.

Halo Exchange: After every epoch, each process shares border rows with neighbours since edge cells depend on values owned by adjacent processes.

```
MPI_Isend / MPI_Irecv → MPI_Waitall
```

Non-blocking communication was chosen over MPI_Sendrecv because the blocking version caused near-deadlock on 4 processes, inflating runtime from ~0.07s to ~6.6s.

Timing: MPI_Wtime() with MPI_Reduce(MPI_MAX) reports the slowest process time.

Correctness: Center cell value gathered to rank 0 and compared against serial results.

5. Analysis

5.1 OpenMP

- On the local machine (2 physical cores), 4 threads are slower than 2 for small grids due to thread overhead dominating computation.
- On HPC (128 cores), OpenMP 4T gives 3.16× speedup for N=1024 (2.717s to 0.860s).
- OpenMP is simple to implement and effective for shared-memory parallelism.

5.2 MPI on HPC

- MPI scales well on HPC hardware with 128 available cores per node.
- For N=1024, speedup with 8 processes is 6.4× (2.717s to 0.424s) — near-linear.
- MPI 8P for N=512 gives 5.9× speedup over serial.
- Performance improves consistently as processes increase, unlike the local machine.

5.3 Local vs HPC Comparison

- Local MPI 4P was slower than serial due to single-machine IPC overhead.
- HPC MPI 4P is 3.2× faster than serial — real hardware benefits are clear.
- HPC has 128 cores vs local 2 cores, so parallelism is actually utilised.

5.4 Performance Bottlenecks

1. MPI_Sendrecv deadlock (resolved): Blocking halo exchange caused 6.6s runtime for 4 processes on the local machine. Fixed with non-blocking calls.
2. Single-machine MPI overhead (local): MPI inter-process communication on one machine adds overhead without extra hardware benefit.

3. Small grid parallelism overhead: For $N=256$, thread/process creation overhead exceeds computation benefit for both OpenMP and MPI.

4. MPI daemon missing on compute nodes: Theorted MPI daemon was not installed on compute nodes, preventing true multi-node MPI execution. All MPI runs were performed on the master node using its 128 cores.

6. Conclusion

The Laplace solver was successfully implemented in Serial, OpenMP, and MPI. Correctness was verified across all implementations for all grid sizes.

On HPC hardware, MPI with 8 processes achieved $6.4\times$ speedup for large grids, demonstrating effective parallel scaling. OpenMP provided $3.16\times$ speedup with 4 threads on the same hardware.

The project is packaged with CMake for easy compilation on any system. The main lesson is that MPI benefits are realised on real multi-core/multi-node hardware — on a single laptop it adds overhead without benefit, but on HPC with 128 cores per node it scales near-linearly.

References

1. LLNL HPC Tutorials: <https://hpc.llnl.gov/documentation/tutorials/introduction-parallel-computing-tutorial>
2. NUST SINES HPC: <https://sines.nust.edu.pk/facilities/screc/>
3. MPI Forum: <https://www.mpi-forum.org/>
4. OpenMP Specification: <https://www.openmp.org/>

2.2 Node Specifications

Node	CPU	Cores	RAM	Uptime
master	AMD EPYC 7452	128	125 GB	head node
compute1	AMD EPYC 7452	128	377 GB	76 days
compute2	AMD EPYC 7452	128	125 GB	112 days
compute3	AMD EPYC 7452	128	125 GB	112 days
compute4	AMD EPYC 7452	128	125 GB	112 days

2.3 Node Availability at Time of Access

Node	Load Average	CPU Usage	RAM Available	Status
compute1	58.24	46.5%	323 GB	BUSY
compute2	0.16	0.0%	96 GB	FREE
compute3	56.55	40.2%	102 GB	BUSY
compute4	30.32	19.5%	93 GB	MODERATE

Availability Ranking: compute2 > compute4 > compute3 > compute1

compute2 was selected for running benchmarks as it had near-zero load.

4. Results

4.1 Correctness Verification

All implementations produce identical center values confirming correctness.

Implementation	N=256 Center	N=512 Center	N=1024 Center
Serial	1.00126e-06	1.18455e-28	1.39099e-57
OpenMP 4T	1.00126e-06	1.18455e-28	1.39099e-57
MPI 8P	1.00126e-06	1.18455e-28	1.39099e-57

4.2 Local Machine Results

Implementation	N=256	N=512	N=1024
Serial	0.064s	0.258s	3.460s
OpenMP 2T	0.037s	0.171s	2.833s
OpenMP 4T	0.109s	0.263s	2.670s
MPI 2P	0.044s	0.157s	2.884s
MPI 4P	0.071s	0.315s	3.599s

4.3 HPC Results (SINES — AMD EPYC 7452, 128 cores)

Implementation	N=256	N=512	N=1024
Serial	0.095s	0.377s	2.717s
OpenMP 4T	0.034s	0.114s	0.860s
MPI 2P	0.058s	0.203s	1.537s
MPI 4P	0.058s	0.109s	0.841s
MPI 8P	0.023s	0.064s	0.424s